2300005633
Ahmet Deniz Sezgin
Report 3

1. Introduction

This report presents the implementation of a client-side monitoring module for the exam platform. The developed module is responsible for identifying the active window and the running processes during an exam session, persisting those observations in a structured format, and preparing the resulting evidence for transmission to the server through the existing communication pipeline.
From an engineering perspective, the requirement is not limited to local collection. The collected data must also remain stable across runtime logging, incident reporting, artifact uploads, and later forensic review. For that reason, the implementation uses JSON and JSONL as the canonical representation for monitoring data and integrates directly with the client session manager, the WebSocket event layer, and the artifact transfer helpers that already exist in the project.

2. System Overview

The monitoring workflow is orchestrated by the client session code in **`client/ws_client.py`**. When the client establishes its WebSocket session, it creates instances of FocusedWindowMonitor and ProcessMonitor and starts them as background components. These monitors run independently from the user interface, continuously collect local state, and write runtime artifacts under the session directory inside **`data/client/<session_uuid>/`**.
The same session object also connects the monitors to the higher-level policy and evidence pipeline. Focused window snapshots are forwarded to **`ClientIncidentEngine.observe_focused_window(...)`**, while process snapshots are forwarded to **`ClientIncidentEngine.observe_processes(...)`**. As a result, the collected data is not only stored for later transmission, but is also evaluated in real time against focused-window and process-blacklist rules enforced by the exam policy.

3. Active Window Detection Module

The active window detector is implemented in **`custommodules/focused_window_monitor/core.py`** together with the Windows-specific collector in **`custommodules/focused_window_monitor/windows.py`**. On Windows, the module reads the foreground window through the **`user32`** API, resolves the owning process identifier, and then enriches the result with **`psutil`** so that the final snapshot includes both window metadata and process metadata.

Each focused-window snapshot can contain fields such as **`window_handle`**, **`window_title, window_class, process_id, process_name,`** and **`process_path`**. The monitor polls once every second and is configured to emit change-oriented evidence rather than identical repeated snapshots. In practice this means it writes an initial record first, then logs **`focused_window_change`** events only when the active window actually changes, which keeps the audit stream compact while still preserving behaviorally significant transitions.

The module can also export a standalone snapshot through **`export_current_snapshot()`**, which produces a point-in-time JSON file when an incident bundle or a manual evidence request is triggered. If the foreground window cannot be obtained or the platform is unsupported, the collector still returns a valid structured object with a reason field and an availability flag. This avoids schema drift and makes server-side handling deterministic.

4. Running Process Detection Module

The running-process detector is implemented in **custommodules/process_monitor/core.py**, with collection helpers in **custommodules/process_monitor/psutil_collector.py** and **custommodules/process_monitor/macos.py**. The default collection path uses **psutil.process_iter(...)** to enumerate processes as lightweight
(pid, name) pairs. Due to testing environment being macOS, a fallback based on the **ps** command is available so that process reporting remains functional even if the primary **psutil-based** path is unavailable.

The process monitor is designed around periodic differencing rather than repeatedly writing full snapshots. It polls every fifteen seconds, computes added and removed sets against the previous observation, and writes diff entries to **processes.jsonl** only when the process state has changed. Every one hundred and twenty seconds it also records a **full_list** snapshot so that the log contains regular full-state anchors in addition to incremental transitions.

When the server sends a **get_processes** request, or when the client prepares incident evidence, the monitor calls **export_requested_report()** to write a full JSON report immediately. The same module also tracks blacklist entries supplied by the exam policy and can detect newly observed blacklisted processes by normalizing process names and comparing them with the current blacklist set. This makes the process monitor both an evidence generator and a local detection component.

5. JSON Output Structure

JSON was selected as the shared data format because it is already the basis of the project protocol implemented in **`common/protocol.py`** and **`common/events.py`**. Every monitoring record is represented as a standard JSON object that contains a timestamp, a type discriminator, and a payload that is specific to the observation being stored. This makes the records easy to serialize, validate, transmit, and archive without requiring a translation layer between local logging and network transport.

In the focused-window stream, entries are written to **`focused_window.jsonl`** with record types such as **`focused_window_initial`**, **`focused_window_change`**, and **`exam_state_marker`**. In the process stream, entries are written to **`processes.jsonl`** with record types such as **`diff`**, **`full_list`**, `requested`, and **`exam_state_marker`**. A focused-window record stores a nested window object, while a process record stores either a processes array for full snapshots or added and removed arrays for **`diffs`**.

This structure is especially suitable for continuous monitoring because JSONL allows each record to remain independently parseable. Tools can read the files sequentially, stream them line by line, or package them into bundles later without reconstructing state from an opaque binary format. The same property also makes incident evidence reproducible, because every recorded event is preserved as a complete self-contained object.

6. Compatibility With the Streaming Module

The monitoring output is compatible with the streaming module because its data model aligns with the project's existing message envelope. Outgoing WebSocket traffic already uses the event-plus-data structure generated by **`common/protocol.py`**, and incident-related monitoring results are wrapped through constructors defined in **`common/events.py`** before being sent by **`client/ws_client.py`**. Since the monitor outputs are already plain JSON objects, they can move into this envelope without structural conversion.

This compatibility is visible in two directions. First, live detections produced from monitor callbacks can be converted into **`incident_report`** messages and transmitted over the WebSocket session immediately. Second, snapshot files created **by `export_current_snapshot()`** and **`export_requested_report()`** can be uploaded through **`upload_runtime_artifact(...)`** or inserted into incident and submission bundles by **`client/transfers.py`**. In both cases, the same logical data shape is preserved from local collection to server-side storage.

As a result, the streaming layer does not need monitor-specific parsing rules beyond ordinary JSON decoding. The transport system only needs to authenticate, wrap, and forward the payload, while the monitoring modules remain responsible for producing stable and semantically meaningful evidence records.

7. Generated Files and Transfer Flow

During an exam session, the client writes continuous monitoring artifacts such as **`data/client/<session_uuid>/focused_window.jsonl` and `data/client/<session_uuid>/processes.jsonl`**. When explicit evidence is needed, it also creates point-in-time files such as **`data/client/<session_uuid>/focused_window_snapshot_<timestamp>.json`** and **`data/client/<session_uuid>/process_report_requested_<timestamp>.json`**. These files are therefore split into continuous audit logs and `requested` snapshot reports, each serving a `different` operational purpose.

The transfer path is already implemented in the existing client stack. When the server requests process evidence, **`client/ws_client.py`** generates a `requested` process report and sends it through **`upload_runtime_artifact(...)`**. When an incident is opened, the client can collect the latest focused-window snapshot, the latest process report, and other runtime evidence into a bundle by using **`build_incident_bundle(...)`**. When the exam ends, related runtime files can also be inserted into the final submission archive by **`build_submission_bundle(...)`**.

This means that the developed monitoring module does not merely log data locally; it produces transport-ready evidence artifacts whose format is already understood by the rest of the system. The combination of JSON-based logs, point-in-time snapshots, WebSocket events, and artifact bundling gives the client both real-time monitoring behavior and reliable post-event traceability.

8. Conclusion

The implemented client-side module satisfies the assignment objective by detecting both the active window and the running processes during the exam and by storing those observations in a structured and transmission-ready form. The design is technically consistent with the rest of the system because the same JSON-oriented representation is used for local logs, incident reporting, artifact uploads, and archived evidence.

Overall, the solution extends the exam platform in a modular and technically robust way. The monitoring logic remains separated into dedicated components, the generated files are suitable for streaming and later review, and the resulting evidence integrates directly with the client-server workflow that is already present in the project.